Mississippi Valley State University

CS 341 — Discrete Structures

Practice Set 48: Approved Solutions

Divide-and-Conquer Recurrences

Covers *MCS* Sections 21.4 and 21.5

Status of this set

Ungraded practice. Solutions released with it. **Always state which case you are in** — that is the graded step.

Part I: How to Practice This Material

The Master Theorem as a competition

For $T(n) = a\,T(n/b) + g(n)$, compute $p = \log_b a$ and compare $g(n)$ with n^p.

Which is bigger	Case	Answer
n^p wins	1	$\Theta(n^p)$
they tie	2	$\Theta(n^p \log n)$
$g(n)$ wins	3	$\Theta(g(n))$

Compute p *first*. Everything follows from comparing it with g.

Three mistakes to expect

1. **Computing $\log_a b$ instead of $\log_b a$.** It is (number of subproblems) in the exponent position.
2. **Forgetting the $\log n$ in Case 2.**
3. **Using the Master Theorem on $f(n-1)$ recurrences.** Those need Wednesday's characteristic equation.

Part II: Practice Problems

Section A — Applying the theorem

1. For each, give $p = \log_b a$, the case, and the solution.

(a) $T(n) = 9T(n/3) + n$

(b) $T(n) = 2T(n/2) + 1$

(c) $T(n) = 8T(n/2) + n^3$

(d) $T(n) = 3T(n/4) + n$

Approved solution

(a) $p = \log_3 9 = 2$. Here $g(n) = n$ grows slower than n^2, so **Case 1**: $T(n) = \Theta(n^2)$.
(b) $p = \log_2 2 = 1$. Here $g(n) = 1$ grows slower than n, so **Case 1**: $T(n) = \Theta(n)$.
(c) $p = \log_2 8 = 3$ and $g(n) = n^3 = \Theta(n^3)$ — a tie, so **Case 2**: $T(n) = \Theta(n^3 \log n)$.
(d) $p = \log_4 3 \approx 0.79$. Here $g(n) = n$ grows faster, so **Case 3**: $T(n) = \Theta(n)$.

(b) and (d) both give $\Theta(n)$ for different reasons — in (b) the recursion dominates, in (d) the combining step does. Naming the case is what shows you know which.

2. Identify the two algorithms and confirm their complexities with the Master Theorem.

(a) $T(n) = T(n/2) + 1$

(b) $T(n) = 2T(n/2) + n$

Approved solution

(a) Binary search. $p = \log_2 1 = 0$, and $g(n) = 1 = \Theta(n^0)$ — a tie, so Case 2 gives $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$. ✓
(b) Merge sort. $p = \log_2 2 = 1$, and $g(n) = n = \Theta(n^1)$ — a tie again, so Case 2 gives $\Theta(n \log n)$. ✓

Both of these were derived twice already — binary search from a tree's height in Lecture 41, merge sort from a diagram in Lecture 45. The Master Theorem gets both from the equation alone. **Both land in Case 2**, which is why that case is the common one in algorithm analysis: a balanced split with linear combining work ties the two sides.

Section B — Which method?

3. For each, say whether the characteristic-equation method or the Master Theorem applies, and why. Do not solve.

(a) $f(n) = 4f(n-1) - 3f(n-2)$

(b) $T(n) = 5T(n/5) + n$

(c) $f(n) = f(n-1) + 2^n$

(d) $T(n) = T(n-1) + T(n/2)$

Approved solution

(a) Characteristic equation. Homogeneous linear, fixed steps back.
(b) Master Theorem. Divide and conquer: $a = 5$, $b = 5$, $g(n) = n$.
(c) Characteristic equation, inhomogeneous version. It steps back one, with $g(n) = 2^n$ — use the five-step method and guess a particular solution of the form $c \cdot 2^n$.

(d) Neither. It mixes a fixed step back with a halving step, so it fits no standard form. (The Akra–Bazzi formula handles some such cases; it is beyond this course.)

Part (d) is the honest one. Not every recurrence yields to a named method, and recognising that is better than forcing the wrong tool.

Section C — The comparison

4. Towers of Hanoi satisfies $T(n) = 2T(n-1) + 1$ and merge sort satisfies $T(n) = 2T(n/2) + n$. Both split into two subproblems.

(a) Which does more work per recursive call?

(b) Which has the smaller solution, and by how much at $n = 64$?

(c) What does the comparison show?

Approved solution

(a) Merge sort, by a wide margin: n units against 1.
(b) Merge sort, overwhelmingly. At $n = 64$: Hanoi needs $2^{64} - 1 \approx 1.8 \times 10^{19}$ steps; merge sort needs about $64 \cdot 6 = 384$ comparisons.
(c) The size of the subproblems dominates everything else. Hanoi's subproblems have size $n - 1$ — barely smaller — while merge sort's have size $n/2$. Doing far more work per call is irrelevant beside that.

Confirmed by the Hanoi variant: raising the per-call cost from 1 to n changes the answer from $2^n - 1$ to $2 \cdot 2^n - n - 2$ — a factor of two, nothing more.

Section D — Writing

5. In five to eight complete sentences, explain why the Master Theorem compares $g(n)$ with $n^{\log_b a}$ rather than with n or with a. What does $n^{\log_b a}$ represent?

Approved solution

Model answer. Unrolling the recurrence produces a tree of subproblems: the top level has one problem of size n, the next has a problems of size n/b, the next a^2 of size n/b^2, and so on. The recursion bottoms out after about $\log_b n$ levels, at which point the number of leaves is $a^{\log_b n}$, and that quantity equals $n^{\log_b a}$. So $n^{\log_b a}$ is exactly the work done at the bottom of the tree, by the base cases alone, with no combining work counted at all. Meanwhile $g(n)$ measures the combining work at the top level. The theorem compares the two because the total is dominated by whichever end of the tree carries more work, and when they balance, every one of the $\log_b n$ levels contributes comparably — which is where the extra factor of $\log n$ in Case 2 comes from.

Full credit requires identifying $n^{\log_b a}$ as the leaf count, and explaining the $\log n$ in Case 2 as the number of levels. That second point is what distinguishes understanding from memorisation.

Instructor notes

- **Problem 1(b) and 1(d) both give $\Theta(n)$** by different cases. Requiring the case name stops students from pattern-matching answers.
- **Problem 3(d) has no standard method.** Include it deliberately — students should learn to say so rather than force a tool.
- **Problem 4 is the block's punchline** and worth the board time: subproblem size beats work per call, by an astronomical margin.
- **Problem 5** explains where $n^{\log_b a}$ comes from. Students who can say "leaf count" understand the theorem; the rest have memorised three cases.